

Optimal BGP Route Reflector Placement in ISP Networks

Applying Facility Location Optimization to a Real Networking Problem

Team Members

Aayush Patel

Dhwani Patel

Course: CPSC 4110/5110 Algorithms in Facility Location

University of Lethbridge

April 2026

Table of Contents

1. Introduction	3
1.1 What We Did	3
2. Existing Solutions and Where Our Approach Fits	5
2.1 BGP Optimal Route Reflection (RFC 9107)	5
2.2 BGP Add-Path (RFC 7911)	5
2.3 Where Our Approach Fits	5
3. Problem Formulation	6
3.1 Mapping to the p-Median Problem	6
3.2 The Math	6
4. Solution Methods	7
4.1 Integer Linear Programming (Exact)	7
4.2 Greedy Forward Selection	7
4.3 Local Search (Swap Heuristic)	7
4.4 Naive Baselines (What Engineers Do Today)	7
5. Data	9
5.1 Canadian Networks	9
6. Results	12
6.1 How Close Are the Heuristics to Optimal?	12
6.2 How Fast Are They?	13
6.3 How Many RRs Do You Actually Need?	16
6.4 Does Optimization Actually Beat What Engineers Do?	16
7. Discussion and Conclusions	20
7.1 What Worked	20
7.2 Comparison with Existing Solutions	20
7.3 Diminishing Returns	20
7.4 Limitations	21
8. Team Contributions	21
9. References	21

1. Introduction

If you run a large ISP network with, say, 200 routers, those routers need to talk to each other to figure out the best way to route traffic. The standard protocol for this is BGP (Border Gateway Protocol), and internally it requires every router to maintain a direct session with every other router. That is a full mesh, and for 200 routers, it means 19,900 simultaneous sessions. It does not scale.

Route Reflectors (RRs) fix the scaling problem. Instead of every router talking to every other router, you pick a few routers as hubs. Everyone else just talks to the hub, and the hub reflects routes to everyone. This brings sessions down from thousands to a handful. Every major ISP in the world uses this approach, including AT&T, Comcast, Bell Canada, TELUS, and Deutsche Telekom.

But here is the catch, When the Route Reflector receives multiple possible routes to the same destination (say, two different exit points out of the network), it has to pick one to tell its clients about. The way BGP works, the RR picks the exit point closest to itself. Not closest to the client who will actually use the route. The client never even hears about the other option.

For example : imagine a Route Reflector sitting in Toronto. It sees two exits to reach Google, one in Toronto and one in Vancouver. It picks Toronto because that is closest to *itself*. But a client router in Calgary, which is way closer to Vancouver, is told to use the Toronto exit. That client is now sending traffic 3,400 km across the country when there was a perfectly good exit 700 km away. This is called the **hot potato routing problem**, and it is documented in RFC 9107.

The networking community is aware of this problem. There are protocol-level solutions: **BGP-ORR** (RFC 9107) lets the RR virtually pretend it is at the client's location when computing paths, and **BGP Add-Path** (RFC 7911) lets the RR send multiple paths so clients can choose their own best exit. Cisco, Juniper, and Nokia all support these features. But both come with significant tradeoffs in CPU cost, memory, and deployment complexity. And critically, even with BGP-ORR, someone still has to decide *where* to place the virtual location, which brings us back to a placement optimization problem.

What nobody has done, as far as we can find, is apply rigorous mathematical optimization to the placement question itself and test it on a scale on real topologies. That is what this project is about. We are not claiming that no solutions exist. We are asking: if you want to optimize RR placement using facility location methods, how well does it work, and how much better is it than what engineers do by intuition?

1.1 What We Did

We formulated the RR placement problem as a p-median facility location problem (directly from the course) and solved it using three methods: Integer Linear Programming for exact solutions, Greedy forward selection for speed, and Local Search with swap moves for the best balance. We tested all three methods on 261 real ISP network topologies from the Internet Topology Zoo, ranging from 4 to 754 routers. We compared the results against naive heuristics that network engineers commonly use (picking high-degree or high-centrality nodes).

The results were pretty striking. **Local Search found the provably optimal solution in 99.9% of cases.** And the naive heuristics that engineers use - They produce routing that is typically **79% worse (Assuming naïve placements)** than what optimization finds. That is a lot of wasted bandwidth and unnecessary latency that real ISPs are dealing with right now.

2. Existing Solutions and Where Our Approach Fits

Before diving into our approach, it is important to understand what solutions already exist for the RR suboptimality problem. The industry has not ignored this issue.

2.1 BGP Optimal Route Reflection (RFC 9107)

BGP-ORR, standardized in 2021, modifies the RR's path selection process. Instead of computing IGP costs from the RR's own location, the RR can be configured with a *virtual location* (an IGP node address) and compute paths from that location's perspective. The operator can configure one virtual location per client group, per client, or for all clients. Cisco IOS-XR, Juniper Junos, and Nokia SR OS all support this feature.

The tradeoffs are real though. Each virtual location requires the RR to run a separate SPF (Shortest Path First) calculation, which increases CPU load. The operator still has to manually decide what the virtual location should be for each group of clients, essentially a placement problem. And deploying BGP-ORR requires software upgrades on the route reflectors, which in a production ISP network is not a trivial change.

2.2 BGP Add-Path (RFC 7911)

Add-Path takes a different approach entirely. Instead of having the RR pick one best path, it sends multiple paths (with path identifiers) to each client, letting clients make their own best-path decision. This completely eliminates the suboptimality problem because every client sees all the options.

The downside is scale. A full Internet routing table has over 1 million prefixes. Sending 2-3 paths per prefix to every client means 2-3x the memory usage on every router in the network. For large ISPs, this can be prohibitively expensive. RFC 9107 explicitly notes that BGP-ORR reduces the amount of state compared to Add-Path for achieving hot potato routing.

2.3 Where Our Approach Fits

Our contribution is complementary to these protocol-level solutions, not a replacement. We address a different question: given that you are deploying Route Reflectors (with or without ORR), where should you physically place them to minimize routing suboptimality?

This matters for several reasons. First, most ISPs today still use standard route reflection (RFC 4456) without ORR or Add-Path. For these networks, physical placement is the only lever available. Second, even with BGP-ORR, the operator needs to choose virtual locations for client groups, and our optimization could inform that choice. Third, our approach requires zero software upgrades, zero protocol changes, and zero additional memory. It is purely a network planning tool.

From a course perspective, this is also where the project connects to facility location theory. The placement question maps directly to the p-median problem, and we can apply the ILP, greedy, and local search methods from the course lectures.

3. Problem Formulation

3.1 Mapping to the p-Median Problem

The connection between RR placement and facility location is pretty direct. Here is how the concepts map:

Course Concept	Our Problem	What It Means
Facility	Route Reflector	The hub that picks routes for clients
Client / demand point	Client router	A router that needs routing info
Candidate location	Any router in the network	Every node could be an RR
Cost to minimize	Routing suboptimality	How much extra distance traffic travels
Parameter p (or k)	Number of RRs	How many hubs to deploy

Table 1: Mapping between facility location concepts from the course and our networking problem.

3.2 The Math

Input: We have a network graph with n routers, weighted edges (we use geographic Haversine distance as a proxy for IGP link costs), exit points (degree-1 stub routers that connect to other networks), and a number k of RRs to place.

Cost Matrix: For every pair (client i , candidate RR location j), we compute $\text{cost}(i,j)$ which measures how much extra distance client i would travel because the RR at j picked the wrong exit. We do this by looking at pairs of exit points: the RR picks whichever is closer to itself, and the client's penalty is the difference between that choice and what the client would have picked on its own.

Objective: Minimize total routing suboptimality:

$$\min \sum_i \sum_j \text{cost}(i,j) \times x_{ij}$$

Subject to: (1) Exactly k facilities open. (2) Each client assigned to exactly one facility. (3) Clients can only be assigned to open facilities. This is the standard p-median ILP from the course lectures.

4. Solution Methods

We implemented three methods from the course. Each has different tradeoffs between solution quality and speed.

4.1 Integer Linear Programming (Exact)

We coded the p-median ILP formulation using JuMP (Julia's optimization framework) with the HiGHS solver, which is free and does not need a license. This gives the provably optimal answer. The catch is speed: since p-median is NP-hard, solve times blow up as the network gets bigger. In our tests, it handled networks up to about 150 nodes fine (under 4 seconds), but anything bigger would take too long. That is the whole reason we need heuristics.

4.2 Greedy Forward Selection

The greedy approach builds a solution one RR at a time. First, it tries every possible node as the sole RR and picks the one that gives the lowest total cost. Then it asks: given that first RR, which second node reduces the remaining cost the most? And so on until we have k RRs.

It is very fast, basically instant even on 754-node networks. But it has a fundamental weakness: it cannot undo bad early decisions. If the first pick turns out to be suboptimal in combination with later picks, there is no way to fix it. In our experiments, greedy found the optimal solution 63% of the time, but when it missed, it was off by an average of 18.1%. Not terrible, but not great either.

4.3 Local Search (Swap Heuristic)

Local search starts with k randomly placed RRs and then tries to improve. At each step, it considers every possible swap: take out one current RR, put in a different node. If the swap improves the objective, accept it. Keep going until no improving swap exists.

To avoid getting stuck in a bad local optimum, we run this 5 times with different random starting points and keep the best result. This turned out to be remarkably effective. Local Search found the provably optimal solution in 1,037 out of 1,038 test cases (99.9%). It missed once, by just 6.3%. And it runs in under 50 milliseconds even on the largest network.

4.4 Naive Baselines (What Engineers Do Today)

To answer the question of whether optimization actually matters, we need something to compare against. We implemented three baselines that represent how network engineers typically place RRs in practice:

Highest Degree: Pick the k most-connected routers. The logic is that well-connected nodes can reach everything quickly. Sounds reasonable, right?

Betweenness Centrality: Pick the k nodes that lie on the most shortest paths between other nodes. This is a measure of how central a node is in the graph.

Random: Pick k nodes at random, do it 20 times, keep the best one. This is our sanity check, the baseline even a coin flip could beat.

Spoiler: *all three of these turn out to be significantly worse than the optimized placement.* We will show exactly how much worse in Section 6.

5. Data

We used the Internet Topology Zoo, a dataset of 261 real ISP network topologies maintained by the University of Adelaide. These are not synthetic or randomly generated networks. They are actual network maps published by ISPs, parsed from official documentation. The dataset includes networks from North America, Europe, Asia, South America, and Africa, covering everything from small regional ISPs to global Tier-1 carriers.

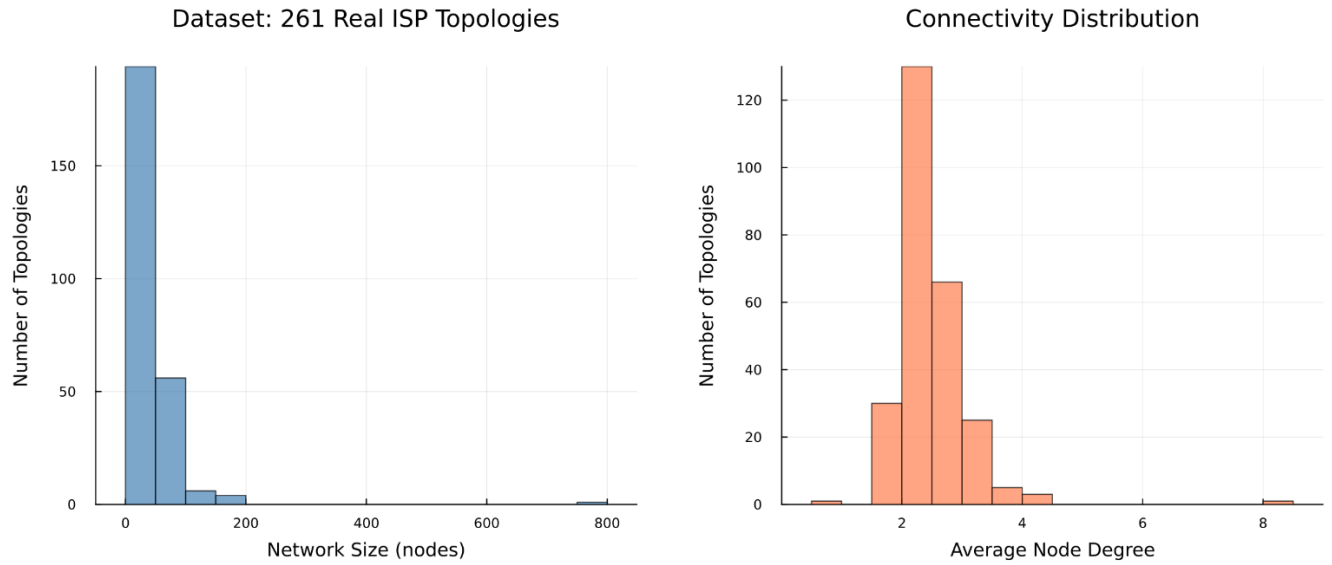


Figure 1: Our dataset of 261 real ISP topologies. Most are small to medium (under 100 nodes), with a few large networks up to 754 nodes.

Each topology file (GML format) contains router locations with latitude and longitude coordinates, plus the physical links between them. Since most files do not include explicit link costs, we use Haversine geographic distance between routers as a proxy for IGP (OSPF/IS-IS) weights. This is a standard approach in the networking literature and makes sense because longer fiber runs generally mean higher latency and cost.

For exit points (routers that peer with other autonomous systems), we use degree-1 stub nodes. If a topology has fewer than 3 of these, we add random nodes up to 20% of the network to ensure a meaningful optimization problem.

5.1 Canadian Networks

We want to highlight a couple of Canadian networks from the dataset.

Bell Canada (48 nodes) spans from Cold Lake and Grande Prairie in Alberta all the way to the Maritime provinces. With $k=3$ RRs, the greedy algorithm placed them in Victoria, Red Deer, & Dallas splitting the network into western, central/eastern, and Pacific clusters.

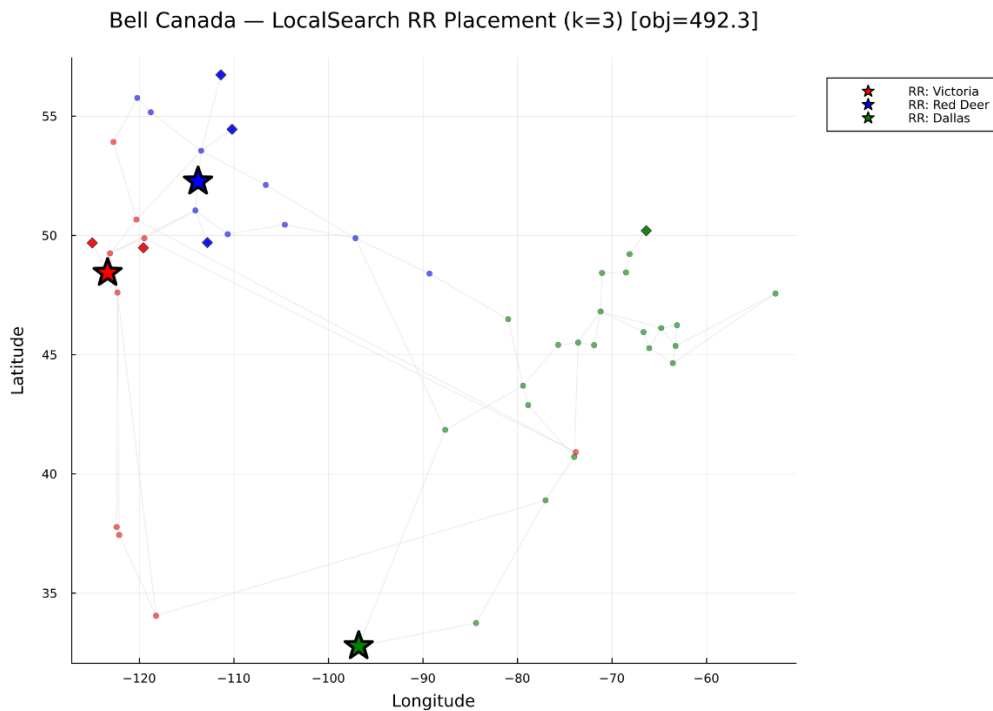


Figure 2: Bell Canada with 3 RRs. Victoria, Red Deer & Dallas

CANARIE (32 nodes) is Canada's national research and education network, the backbone that connects Canadian universities. The algorithm placed RRs in Kamloops (West), Thunder Bay (Central), and Halifax (East), providing clean coast-to-coast coverage.

Canerie — LocalSearch RR Placement ($k=3$) [obj=2304.2]

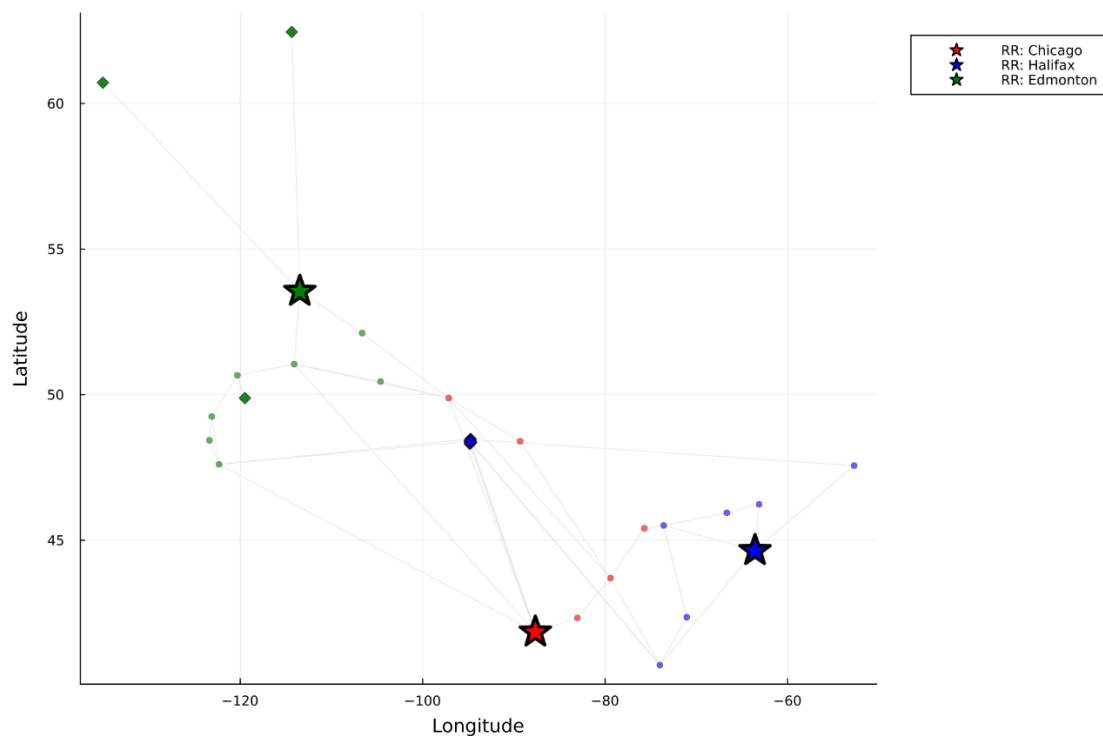


Figure 3: CANARIE research network with 3 RRs providing coast-to-coast coverage across Canada.

For comparison, here is **Cogent (197 nodes)**, a global Tier-1 ISP spanning North America and Europe. With $k=5$, the algorithm placed RRs in San Diego, Indianapolis, Raleigh, Liverpool Southport, and Vienna, covering both continents with clear geographic clusters.

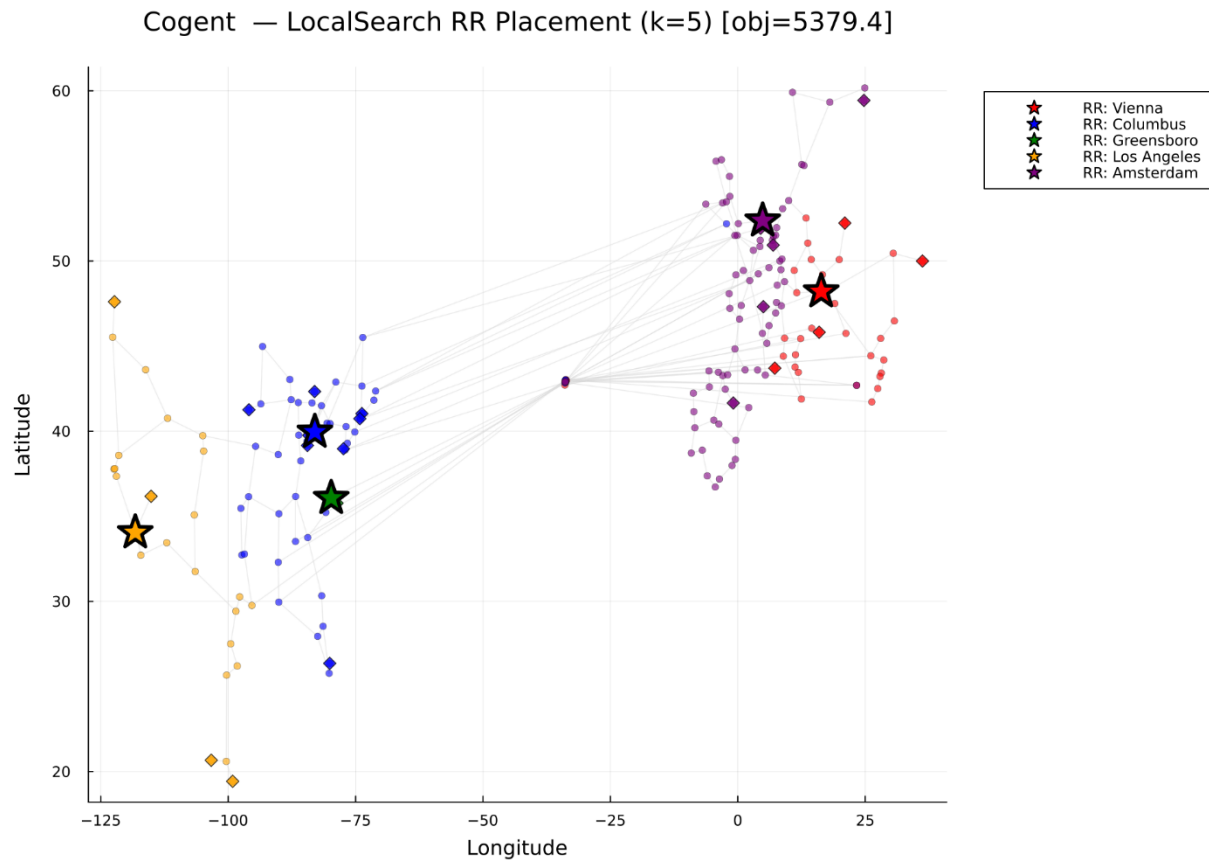


Figure 4: Cogent global network with 5 RRs. Each color represents the clients assigned to that RR.

6. Results

We ran five experiments across all 261 topologies. The full experiment suite completed in about 75-120 seconds on a standard laptop. Here are the actual numbers.

6.1 How Close Are the Heuristics to Optimal?

We ran the ILP (exact solver) on every topology up to 150 nodes, across $k = 1, 2, 3$, and 5. That gave us 1,038 instances where we know the true optimal solution. Then we measured how far greedy and local search deviated from it.

Method	Found Optimal	Rate	Avg Gap When Not
Local Search	1,037 / 1,038	99.9%	6.3%
Greedy	654 / 1,038	63.0%	18.1%

Table 2: Optimality comparison across 1,038 test instances.

Local Search is essentially perfect. Greedy is decent but unreliable. The histogram below shows this clearly: Local Search is a spike right at 0% (always finds optimal), while Greedy has a long tail stretching out to 200%, meaning there are topologies where it is really bad.

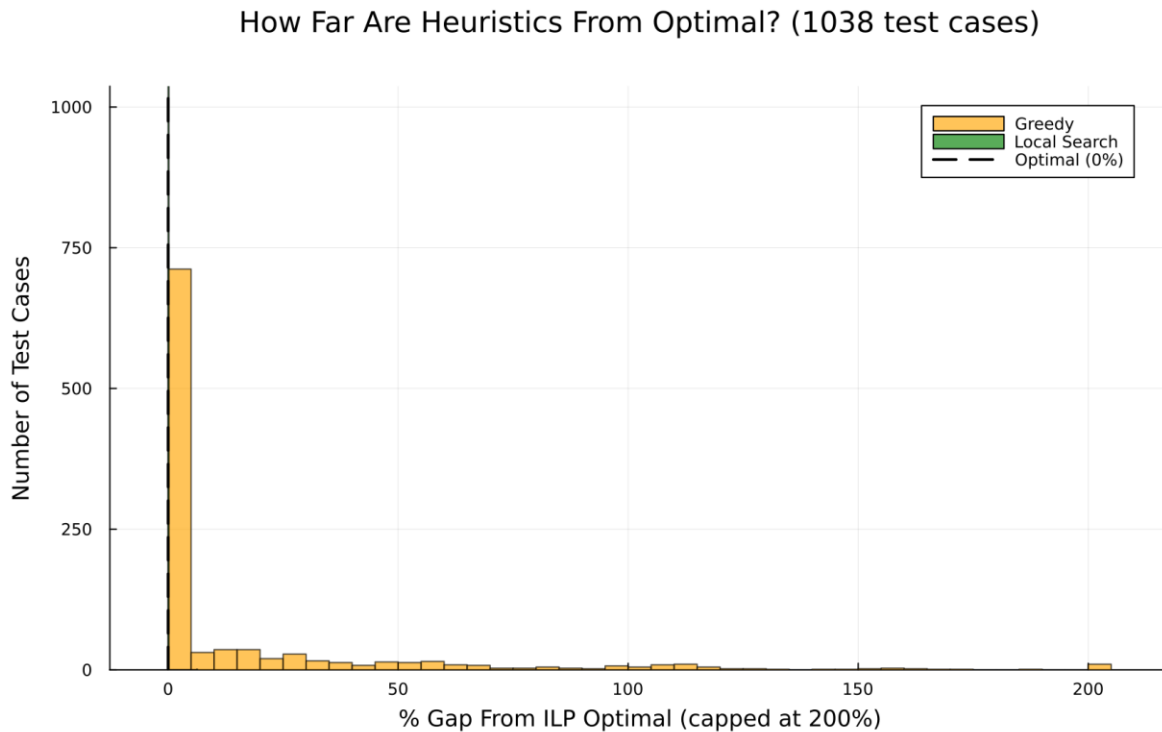


Figure 5: How far are heuristics from optimal? Local Search (green spike at 0%) vs Greedy (orange spread).

Why do we even need heuristics? Because the ILP does not scale. Look at how solve time grows with network size:

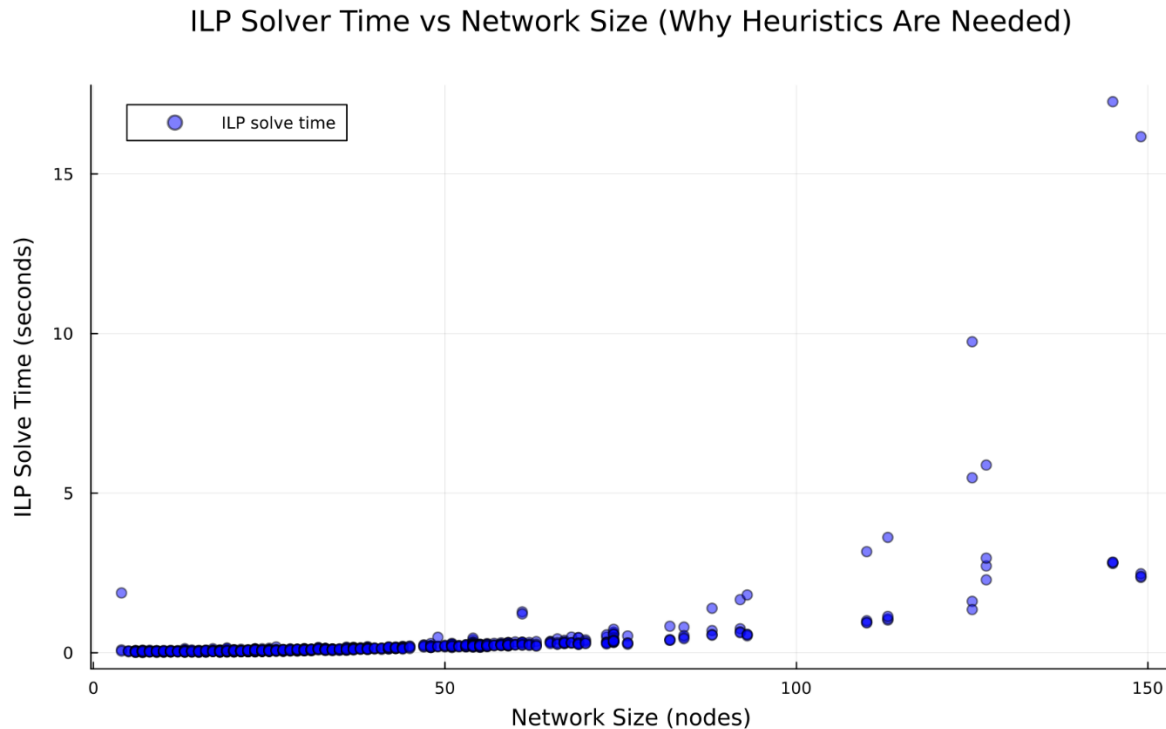


Figure 6: ILP solve time vs network size. Grows exponentially, reaching ~4 seconds at 150 nodes.

At 150 nodes it already takes 4 seconds. Extrapolating that curve, a 500-node network would take minutes, and a 1000-node network could take hours. Real ISP backbones have hundreds to thousands of routers. The ILP is great for proving our heuristics are correct, but it is not a practical tool for large networks.

6.2 How Fast Are They?

We ran all three methods on every topology with $k=3$. The speed difference is dramatic:

Method	Avg Runtime	Max Runtime	Topologies
ILP	0.081 sec	1.09 sec	256 (up to 150 nodes)
Greedy	< 0.001 sec	0.008 sec	All 261
Local Search	0.001 sec	0.041 sec	All 261

Table 3: Runtime comparison. Greedy and Local Search are practically instant.

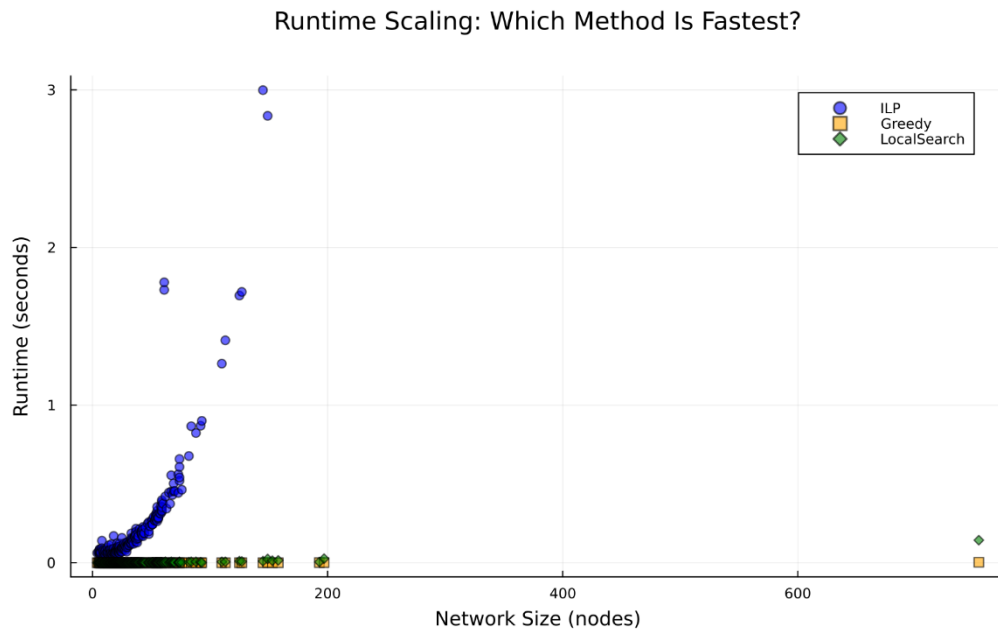


Figure 7: Runtime scaling across all 261 topologies. ILP (blue) blows up; Greedy and Local Search stay flat near zero.

We also did a direct head-to-head comparison of Greedy vs Local Search on every topology. Each dot in the plot below is one network. If the dot is above the red diagonal line, Greedy produced a worse solution:

Greedy vs Local Search (each dot = 1 topology)

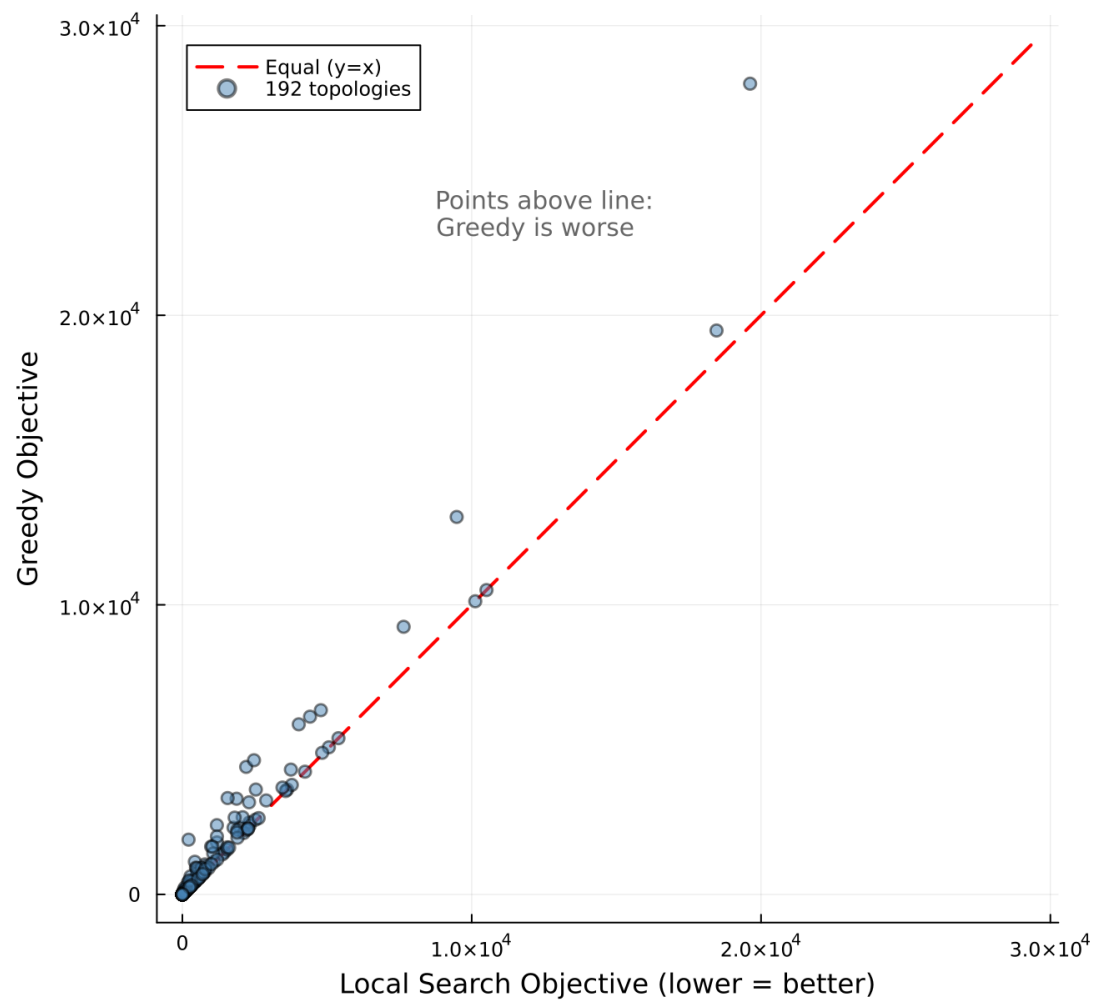


Figure 8: Greedy vs Local Search, head-to-head on 192 topologies. Nearly every dot is above the line.

Almost every dot is above the line. Local Search wins consistently. The few dots on the line are small networks where both methods find the same optimal solution.

6.3 How Many RRs Do You Actually Need?

This is a practical question any network architect would ask. We tested three networks of different sizes (GEANT with 40 nodes, ION with 125, Cogent with 197) and varied k from 1 to 20.

The answer: the first few RRs help enormously, but after that you hit diminishing returns. Going from 1 RR to 2 eliminates 55-93% of the routing penalty. By $k=5$, less than 15% of the original problem remains. After $k=10$, you are down to under 5%, and adding more barely moves the needle.

How Much Does Each Additional RR Help?

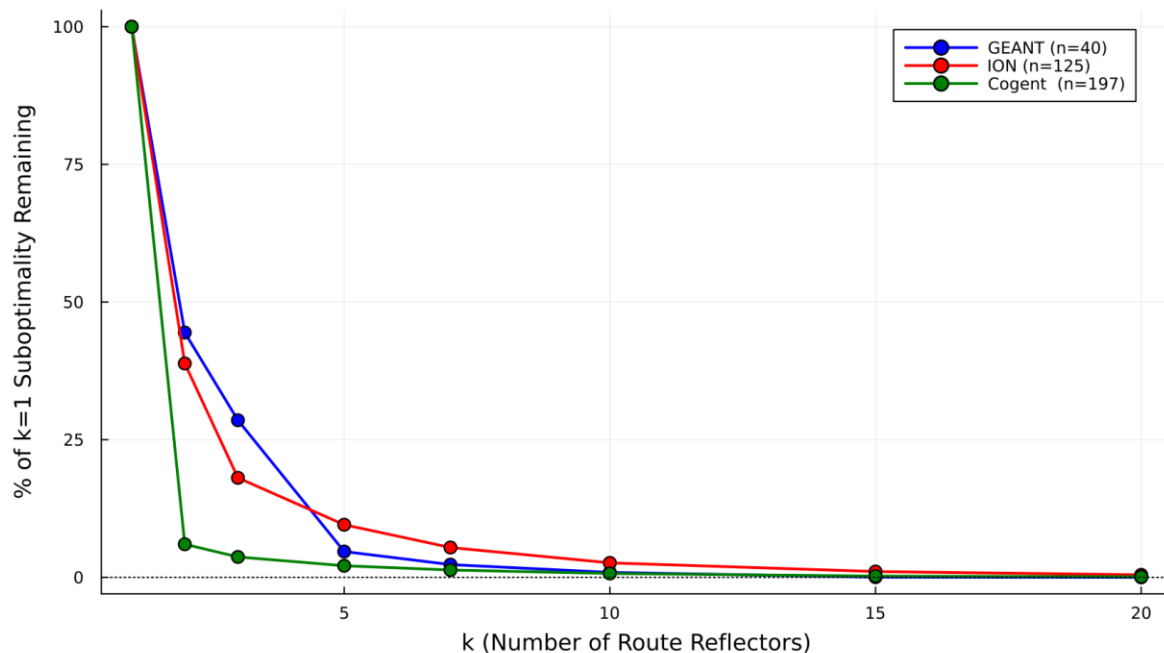


Figure 9: Each line shows how quickly suboptimality drops as RRs are added. By $k=10$, over 95% of the problem is solved.

Practical takeaway: For a network of 40-200 routers, 3-5 well-placed RRs capture the vast majority of the benefit. There is little reason to deploy more than 10 unless you have very specific operational requirements.

6.4 Does Optimization Actually Beat What Engineers Do?

This is the big question. Network engineers have been placing Route Reflectors for 25+ years based on rules of thumb: pick the most connected router, pick the most central one, or just pick whatever seems reasonable. **Does mathematical optimization actually do better?**

The answer is a resounding yes. Here are the numbers at $k=3$ across 192 topologies:

Method	Median Gap	Avg Gap	Wins	Type
Local Search	0.0%	0.0%	260/261	Optimized
Greedy	0.0%	12.3%	137/261	Optimized
Random (best of 20)	5.4%	16.6%	89/261	Naive
Betweenness	69.4%	514%	60/261	Naive
Highest Degree	79.3%	665%	63/261	Naive

Table 4: Method comparison at $k=3$. Naive methods are typically 70-80% worse than optimized placement.

Look at those numbers. Highest Degree placement, which is probably the most common approach in industry, produces a solution that is **median 79.3% worse** than what Local Search finds. Betweenness centrality is almost as bad at 69.4%. Even random placement (best of 20 tries!) only has a median gap of 5.4%.

Here is what this means in real terms: an ISP using rule-of-thumb RR placement is forcing roughly 79% more traffic through suboptimal paths than necessary. That is wasted bandwidth, higher latency, and worse user experience, all because nobody ran a simple algorithm that takes 40 milliseconds.

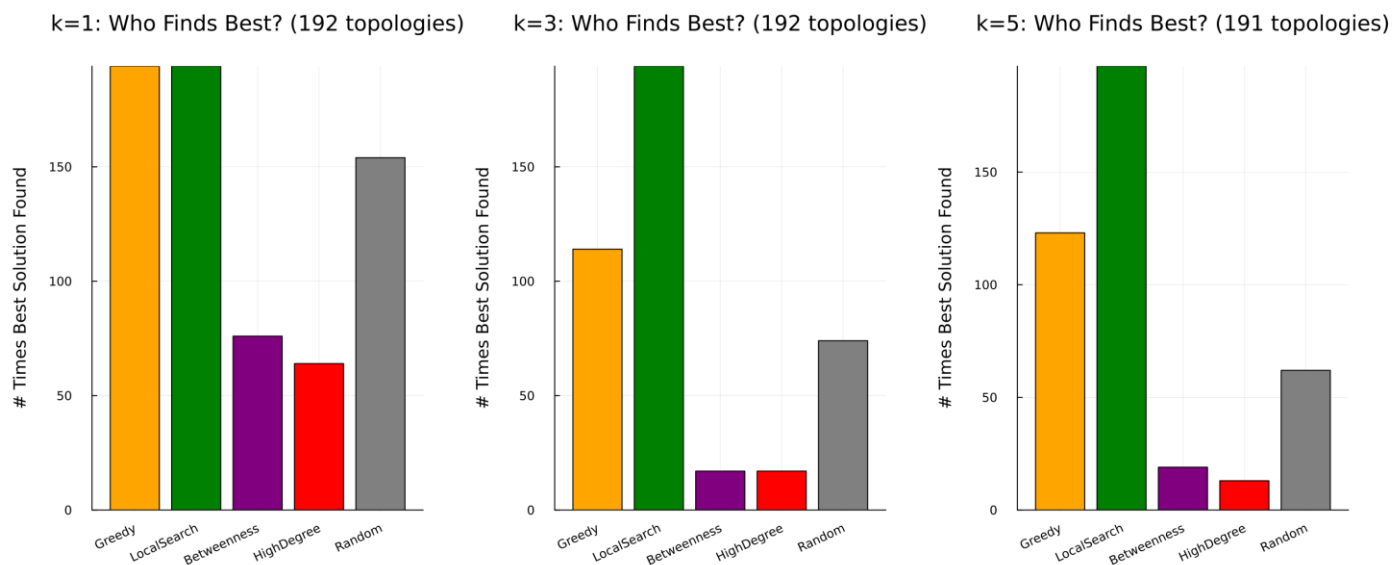


Figure 10: How often each method found the best solution. Local Search (green) dominates at every k value.

The strip plot below shows the full distribution for each method. Each dot is one topology. Look at how tightly Local Search clusters at 0%, while Betweenness and HighDegree are scattered wildly up to 500%+:

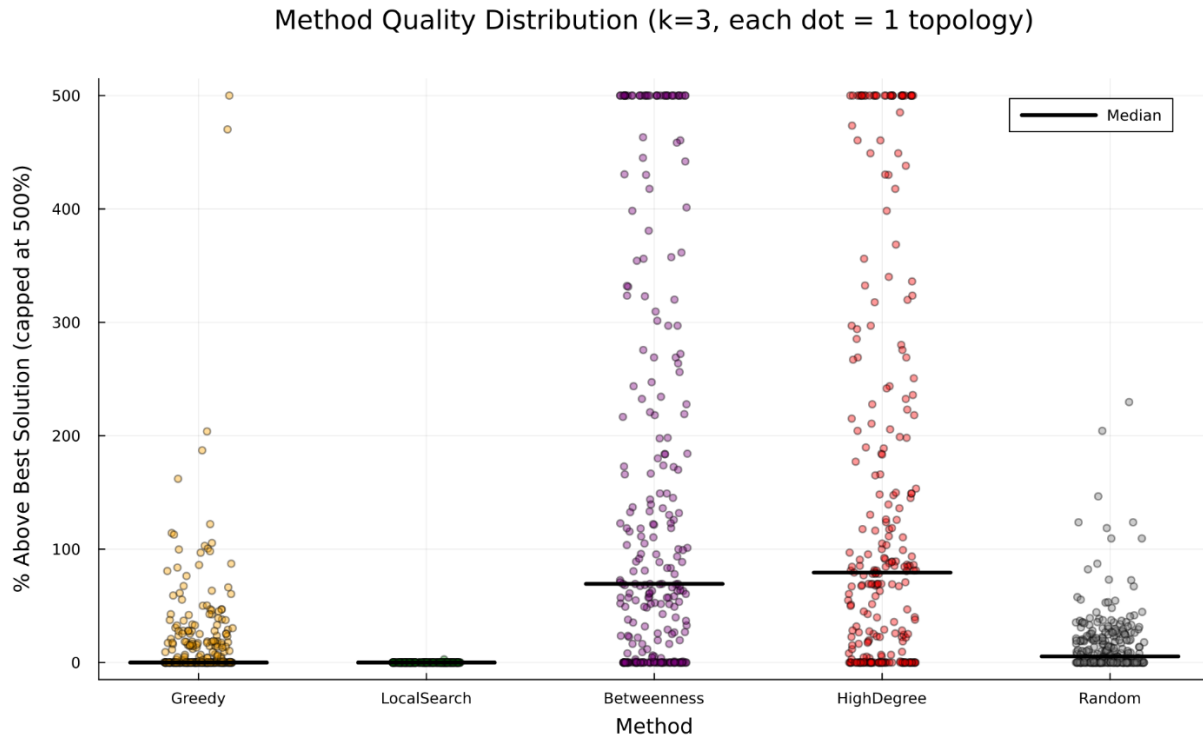


Figure 11: Full quality distribution at k=3. Each dot = one topology. Local Search (tight at 0%) vs naive methods (scattered up to 500%+).

And here is the most direct visualization: the before-and-after comparison. The left scatter plot shows every topology with the naive (highest degree) cost on the y-axis and the optimized (local search) cost on the x-axis. Every dot above the diagonal line means optimization helped. The right histogram shows the improvement distribution.

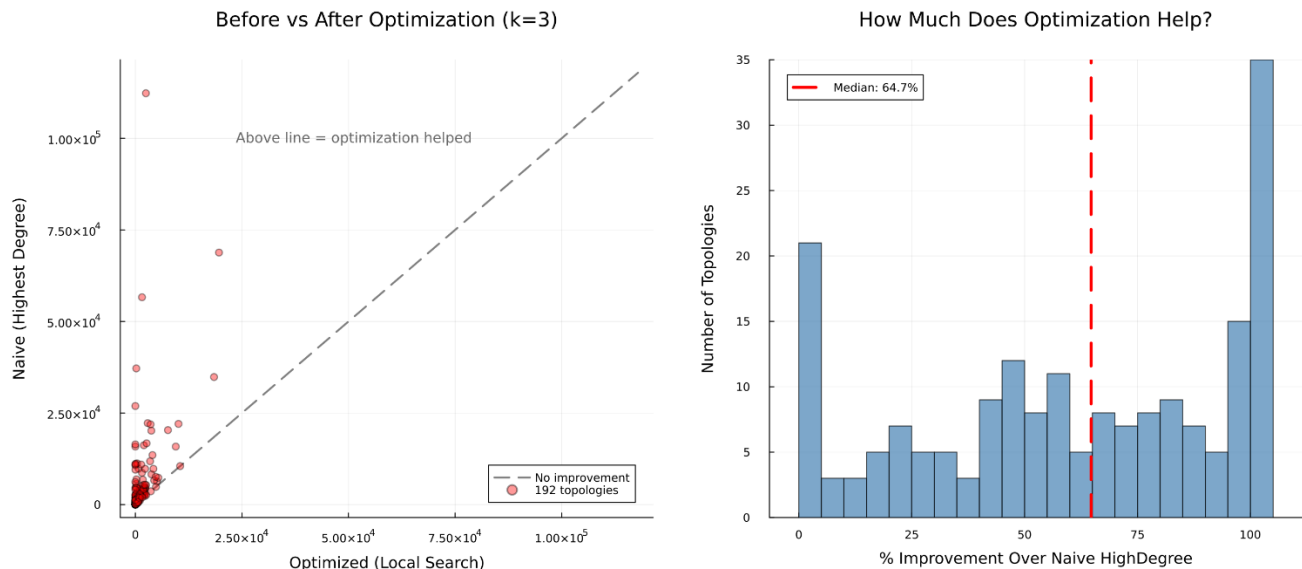


Figure 12: Before vs After. Left: optimization helps on nearly every topology. Right: median improvement is 64.7%.

The median improvement is 64.7%. That means for a typical ISP, switching from intuitive RR placement to optimized placement would reduce routing suboptimality by almost two-thirds.

7. Discussion and Conclusions

7.1 What Worked

Local Search is the clear winner. It found the optimal solution 99.9% of the time, runs in under 50ms on any network size, and needs no commercial solver. If an ISP wanted to deploy this tomorrow, they could. The algorithm is simple (just try swapping RRs and keep improvements).

Greedy is a reasonable fallback. At 63% optimal, it is useful for quick exploratory analysis. But its 18.1% average gap when it misses means you should always follow up with local search if you want a good answer.

ILP provides ground truth but does not scale. Its value is as a benchmark. We used it to prove that our heuristics are good, which would not be possible without it.

7.2 Comparison with Existing Solutions

It is worth being explicit about how our approach compares with the protocol-level solutions discussed in Section 2.

vs BGP-ORR (RFC 9107): ORR eliminates suboptimality by running path selection from the client's perspective. It is the more complete solution if your routers support it. However, it requires software upgrades on all route reflectors, increases CPU load (one SPF computation per client group), and the operator still needs to configure virtual locations for each client group. Our optimization could actually help ORR deployments by informing the choice of virtual locations. For ISPs still on standard route reflection without ORR, our placement optimization is the only option that requires no software changes.

vs BGP Add-Path (RFC 7911): Add-Path is the most complete solution: it sends all paths to all clients, eliminating suboptimality entirely. The cost is memory. With 1M+ Internet prefixes and 2-3 paths each, every router in the network needs significantly more RAM. Our approach achieves zero additional memory overhead because we are only changing which existing routers are designated as RRs, not adding any new BGP state.

vs manual placement: This is where our results are most striking. Most ISPs today use neither ORR nor Add-Path. They place RRs manually, often at high-degree or high-centrality nodes. Our experiments show this produces routing that is median 79% worse than optimized placement. A 40-millisecond local search run can recover most of this penalty.

7.3 Diminishing Returns

Our experiments show that 3-5 RRs capture most of the benefit for networks of 40-200 nodes. Beyond $k=10$, there is almost no improvement left. This gives network architects a concrete guideline rather than guessing.

7.4 Limitations

We want to be upfront about the limitations of this work. First, we used geographic distance as a proxy for IGP link weights. Real networks have manually configured OSPF/IS-IS costs that may differ significantly from geography. Second, our cost model samples exit-point pairs rather than enumerating all actual BGP prefixes, which is a simplification of what happens in production networks. Third, the Topology Zoo data, while real, is somewhat dated (2012).

More fundamentally, our approach optimizes physical placement of standard RRs. If an ISP has the option to deploy BGP-ORR or Add-Path, those protocol-level solutions can provide more complete suboptimality elimination than placement optimization alone. Our approach is most valuable for ISPs that cannot or choose not to upgrade their router software, or as a planning input for ORR virtual location configuration.

That said, the underlying optimization approach (p-median solved via local search) would work identically with real IGP weights and actual prefix data if available, and the 261-topology evaluation gives us confidence that the results generalize.

8. Team Contributions

Member	Contribution	Weight
Aayush Patel	Problem formulation, ILP solver, report writing, Visualization, baseline methods	50%
Dhwani Patel	Greedy and local search, experiments, data processing, testing	50%

9. References

- [1] E. Rosen, Y. Rekhter. *BGP Route Reflection: An Alternative to Full Mesh Internal BGP (RFC 4456)*. IETF, 2006.
- [2] R. Raszuk, B. Decraene, C. Cassar, K. Wang. *BGP Optimal Route Reflection (RFC 9107)*. IETF, 2021.
- [3] D. Walton, A. Retana, E. Chen, J. Scudder. *Advertisement of Multiple Paths in BGP (RFC 7911)*. IETF, 2016.
- [4] S. Knight, H. Nguyen, N. Falkner, R. Bowden, M. Roughan. *The Internet Topology Zoo*. IEEE JSAC, 29(9), 2011.
- [5] Internet Topology Zoo dataset: <https://topology-zoo.org/dataset.html>
- [6] Y. Breitbart, M. Garofalakis, A. Gupta, A. Kumar, R. Rastogi. *On Configuring BGP Route Reflectors*. IEEE COMSWARE, 2007.

- [7] R. Benkoczi. *Introduction to Facility Location Problems*. CPSC 4110/5110 Lecture Notes, University of Lethbridge, 2026.
- [8] R. Benkoczi. *Node Search Algorithms for the p -Center*. CPSC 4110/5110 Course Notes, 2026.
- [9] G. Cornuejols, M. Fisher, G. Nemhauser. *Facility Location*. Integer Programming Tutorial, 1995.
- [10] JuMP.jl: Modeling language for mathematical optimization in Julia. <https://jump.dev/>
- [11] HiGHS: High-performance open-source linear programming solver. <https://highs.dev/>